

```
# EXERCÍCIO 1 - REFERÊNCIAS
```

```
a = [10, 20, 30]  
b = a
```

```
b.append(40)
```

```
print("a:",a)  
print("b:",b)  
print("id(a):", id(a))  
print("id(b):", id(b))
```

```
a: [10, 20, 30, 40]  
b: [10, 20, 30, 40]  
id(a): 140180074138816  
id(b): 140180074138816
```

EXERCÍCIO 1

1. Qual será a saída? R = a: [10, 20, 30, 40] b: [10, 20, 30, 40] id(a): 131946976211776 id(b): 131946976211776 \n\n\n
2. a e b representam o mesmo objeto? R = Sim, eles referenciam o mesmo objeto
3. Por quê? R = Por que a lista original é modificada sem fazer uma nova lista e b passa a ser uma referência para a
4. O que aconteceria com b = a.copy()? R = a saída de a seria 10, 20, 30 e a saída de b 10,20,30, 40 por que cria uma cópia da lista original e depois adiciona o novo elemento

```
# EXERCÍCIO 2 - CONSTRUINDO UMA CADEIA
```

```
class Node:  
    def __init__(self,valor):  
        self.valor = valor  
        self.proximo = None
```

```
n1 = Node("A")  
n2 = Node("B")  
n3 = Node("C")
```

```
n1.proximo = n2  
n2.proximo = n3
```

```
atual = n1  
while atual is not None:  
    print(atual.valor)
```

```
atual = atual.proximo
```

A
B
C

```
# EXERCÍCIO 3 - DEPURAÇÃO
```

```
n1 = Node(10)  
n2 = Node(20)  
n3 = Node(30)
```

```
n1.proximo = n2  
n2.proximo = n3
```

```
atual = n1
```

```
while atual is not None:  
    print(atual.valor)
```

EXERCÍCIO 3

1. Qual é o problema?

R = O código entra em loop infinito

2. Por que o programa não termina?

R = Porque a variável atual nunca é alterada. Ela continua apontando para n1 e não avança para o próximo nó.

3. Qual linha deve ser acrescentada?

R = atual = atual.proximo

4. Qual será a saída depois da correção?

R = 10 20 30

```
# PROBLEMA REAL - SISTEMA DE ATENDIMENTO DE UMA CLÍNICA
```

```
"""
```

```
Amplie o sistema.
```

```
Pacientes com prioridade devem ser atendidos antes dos pacientes normais.
```

```
Teste:
```

```
fila vazia;
```

```
um paciente;
vários pacientes;
atendimento;
remoção do último paciente;
paciente prioritário.
"""

from dataclasses import dataclass

@dataclass
class Paciente:
    nome: str
    idade: int

    def __post_init__(self):
        self.prioridade = self.idade >= 60

class Node:
    def __init__(self, paciente):
        self.paciente = paciente
        self.proximo = None

class FilaAtendimento:
    def __init__(self):
        self.inicio = None
        self.fim = None
        self._tamanho = 0

    def esta_vazia(self):
        if self.inicio is None:
            print("A fila está vazia")
            return True
        else:
            print("Há pacientes na fila")
            return False

    def adicionar(self):
        paciente = Paciente(
            nome = input (f"Nome: "),
            idade = int(input(f"Idade: ")),
        )
        node = Node(paciente) # criando objeto node que recebe paciente
        p = node.paciente

        if self.inicio is None:
```

```
self.inicio = node
self.fim = node
else:
    if p.prioridade:

        # O primeiro paciente é normal
        if not self.inicio.paciente.prioridade:
            node.proximo = self.inicio
            self.inicio = node

        else:
            # Já existem prioritários
            atual = self.inicio

            # Procura o último paciente prioritário
            while (atual.proximo is not None and
                    atual.proximo.paciente.prioridade):
                atual = atual.proximo

            # Insere o novo prioritário depois dos prioritários existentes
            node.proximo = atual.proximo
            atual.proximo = node

            # Se entrou no final, atualiza o fim
            if node.proximo is None:
                self.fim = node

    else:
        self.fim.proximo = node
        self.fim = node

self._tamanho += 1

def listar(self):
    atual = self.inicio

    if atual is None:
        print("A fila está vazia\n")
        return

    while atual is not None:

        p = atual.paciente # acessa o objeto paciente dentro do nó

        if p.prioridade:
            tipo = "Prioridade"
        else:
            tipo = "Normal"

        print(f"Nome:{p.nome} | Idade: {p.idade} | prioridade {tipo}")
```

```
        atual = atual.proximo

def atender(self):
    # atender primeiro paciente

    atual = self.inicio

    if atual is None:
        print("A fila está vazia\n")
        return

    p = atual.paciente
    print(f"Paciente {p.nome} atendido")

    if self.inicio == self.fim:
        self.inicio = None
        self.fim = None
    else:
        self.inicio = atual.proximo

    self._tamanho -= 1

def tamanho(self):

    print(f"Quantidade de pessoas na fila : {self._tamanho}")

def menu():

    while True:
        print ("\n ==== MENU ===\n")
        print("1 - Adicionar paciente")
        print("2 - Listar pacientes")
        print("3 - Atender primeiro paciente")
        print("4 - Verificar se a fila está vazia")
        print("5 - Informar quantidade de pacientes")
        print("6 - Sair")

        opcao = int(input("Escolha uma opção: "))

        if opcao == 1:
            fila.adicionar()
        elif opcao == 2:
            fila.listar()
        elif opcao == 3:
            fila.atender()
```

```
elif opcao == 4:
    fila.esta_vazia()
elif opcao == 5:
    fila.tamanho()
elif opcao == 6:
    print("Programa encerrado")
    break
else:
    print("Opção inválida")
```

```
fila = FilaAtendimento()
menu()
```

```
==== MENU ===
```

```
1 - Adicionar paciente
2 - Listar pacientes
3 - Atender primeiro paciente
4 - Verificar se a fila está vazia
5 - Informar quantidade de pacientes
6 - Sair
Escolha uma opção: 4
A fila está vazia
```

```
==== MENU ===
```

```
1 - Adicionar paciente
2 - Listar pacientes
3 - Atender primeiro paciente
4 - Verificar se a fila está vazia
5 - Informar quantidade de pacientes
6 - Sair
Escolha uma opção: 0
Opção inválida
```

```
==== MENU ===
```

```
1 - Adicionar paciente
2 - Listar pacientes
3 - Atender primeiro paciente
4 - Verificar se a fila está vazia
5 - Informar quantidade de pacientes
6 - Sair
Escolha uma opção: 6
Programa encerrado
```

Fazer uma ordem de prioridade dentro da função adicionar()

Implementar a ordem de prioridade dentro da função adicionar(). A função listar() deverá exibir a fila na ordem de prioridade, que será a ordem utilizada para o atendimento.

TESTE

==== MENU ===

1 - Adicionar paciente 2 - Listar pacientes 3 - Atender primeiro paciente 4 - Verificar se a fila está vazia 5 - Informar quantidade de pacientes 6 - Sair Escolha uma opção: 2 Nome:João | Idade: 69 | prioridade Prioridade Nome:Marcos | Idade: 80 | prioridade Prioridade Nome:Maria | Idade: 22 | prioridade Normal Nome:José | Idade: 11 | prioridade Normal Nome:Ana | Idade: 45 | prioridade Normal

==== MENU ===

1 - Adicionar paciente 2 - Listar pacientes 3 - Atender primeiro paciente 4 - Verificar se a fila está vazia 5 - Informar quantidade de pacientes 6 - Sair Escolha uma opção: 3 Paciente João atendido

==== MENU ===

1 - Adicionar paciente 2 - Listar pacientes 3 - Atender primeiro paciente 4 - Verificar se a fila está vazia 5 - Informar quantidade de pacientes 6 - Sair Escolha uma opção: 4 Há pacientes na fila

==== MENU ===

1 - Adicionar paciente 2 - Listar pacientes 3 - Atender primeiro paciente 4 - Verificar se a fila está vazia 5 - Informar quantidade de pacientes 6 - Sair Escolha uma opção: 5 Quantidade de pessoas na fila : 4

==== MENU ===

1 - Adicionar paciente 2 - Listar pacientes 3 - Atender primeiro paciente 4 - Verificar se a fila está vazia 5 - Informar quantidade de pacientes 6 - Sair Escolha uma opção: 6 Programa encerrado

=====

==== MENU ===

1 - Adicionar paciente 2 - Listar pacientes 3 - Atender primeiro paciente 4 - Verificar se a fila está vazia 5 - Informar quantidade de pacientes 6 - Sair Escolha uma opção: 4 A fila está vazia

==== MENU ===

1 - Adicionar paciente 2 - Listar pacientes 3 - Atender primeiro paciente 4 - Verificar se a fila está vazia 5 - Informar quantidade de pacientes 6 - Sair Escolha uma opção: 0 Opção inválida

==== MENU ===

1 - Adicionar paciente 2 - Listar pacientes 3 - Atender primeiro paciente 4 - Verificar se a fila está vazia 5 - Informar quantidade de pacientes 6 - Sair Escolha uma opção: 6 Programa encerrado

